

Lernziele:

- Code Refactoring
- Inkrementelle Softwareentwicklung und Vertiefung des bisher Gelernten

Vorbemerkung

In eurem Git-Repository sollten (mindestens) die Dateien

`expr.abc`, `expr.hdr`, `lexer.abc`, `lexer.hdr`, `simple.isa`, `xtest_expr.abc`,
`xtest_lexer.abc`, `xtest_parser.abc`

enthalten sein.

Schritt 1 (Makefile verwenden)

Aktuell kann man gerade noch die Übersicht bewahren, wie man die verschiedenen Testprogramme übersetzen kann. Zum Beispiel:

- `xtest_lexer` mit: `abc xtest_lexer.abc lexer.abc`
- `xtest_parser` mit: `abc xtest_parser.abc`
- `xtest_expr` mit: `abc xtest_expr.abc expr.abc`

In dieser Session wird sich aber die Anzahl der Source-Files, Header-Files und Testprogramme erhöhen und es wird unübersichtlicher, wie man jedes Testprogramm übersetzen muss. Paradoxerweise wird sich die Anzahl der Dateien erhöhen, um die Gesamtstruktur des Projekts besser im Überblick behalten zu können:

- Alle Testprogramme beginnen mit `xtest` und enden auf `.abc`
- In allen anderen `.abc`-Dateien wird eine Funktionalität (zum Beispiel Lexer, Parser, etc.) implementiert und in einer zugehörigen Header-Datei (Endung `.hdr`) wird deklariert, welche Datenstrukturen und Funktionen extern zur Verfügung stehen.

Zum Glück kann der Übersetzungsprozess aber mit einem `Makefile` automatisiert werden. Ein für unsere Zwecke passendes `Makefile` findet ihr auf der Vorlesungsseite zu Session 15. Mit dem Kommando `wget` (muss eventuell installiert werden) kann dies direkt ins Projektverzeichnis geladen werden. Kopiert den Link zum `Makefile` und führt dann innerhalb des Projektverzeichnisses `wget` wie folgt aus:

```
wget [link mit Copy&Paste einfügen]
```

Testet danach das `Makefile`:

- Mit `make` sollten alle Testprogramme übersetzt werden.
- Mit `make clean` sollte aus den Source-Files der erzeugte Code wieder gelöscht werden.

Schritt 2 (Etwas Hintergrund zum Makefile)

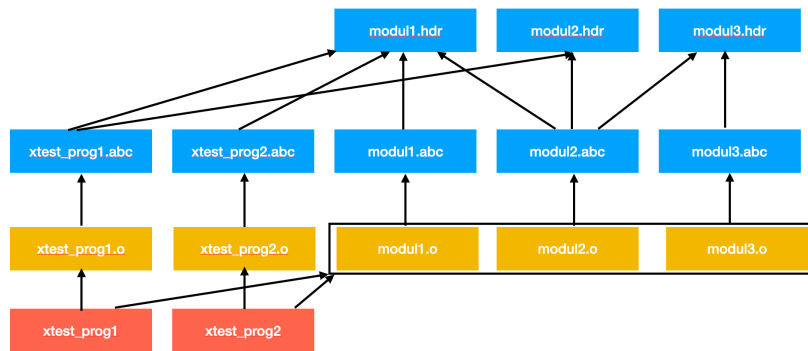
Durch das `Makefile` wird für jedes Source-File ein Object-File erzeugt. Dies geschieht nur dann, wenn einer der folgenden Gründe vorliegt:

- Das Object-File existiert noch nicht.
- Das zugehörige Source-File hat sich geändert.
- Eine mit `@` eingebundene Datei hat sich geändert.

Das `Makefile` generiert auch die Testprogramme, indem es die Object-Files verlinkt. Dabei wird das zum Testprogramm zugehörige Object-File mit allen Object-Files verlinkt, die nicht mit `xtest` beginnen. Auch das Verlinken geschieht nur dann, wenn einer der folgenden Gründe vorliegt:

- Das ausführbare Testprogramm existiert noch nicht.
- Ein benötigtes Object-File hat sich verändert.

Beim Übersetzen werden also Abhängigkeiten berücksichtigt. Solche Abhängigkeiten kann man grafisch wie folgt darstellen. Die Pfeile kann man als "hängt ab von" interpretieren:



Im Makefile werden diese Abhängigkeiten textuell beschrieben. Beim Überprüfen, ob sich eine Datei verändert hat, wird nicht der Datei-Inhalt, sondern der Zeitstempel der letzten Änderung überprüft. Das kann man leicht testen. Mit dem Kommando `touch` kann man den Zeitstempel einer Datei aktualisieren oder eine neue (leere) Datei anlegen, falls diese noch nicht existiert. Der Effekt von `touch xtest_lexer.abc` ist also der gleiche wie wenn man im Editor auf Speichern klickt.

Aufgaben:

1. Führt `make clean` und dann `make` aus.
2. Führt nochmals `make` aus. Die Meldung sollte sein, dass nichts zu tun ist.
3. Führt `touch xtest_parser.abc` gefolgt von `make` aus. Der Effekt sollte sein, dass `xtest_parser.o` im Unterverzeichnis `obj` neu erzeugt wird und dann `xtest_parser` gelinkt wird.
4. Führt `touch lexer.abc` gefolgt von `make` aus. Der Effekt sollte sein, dass `lexer.o` neu erzeugt wird und dann alle Testprogramme neu gelinkt werden.
5. Führt `touch lexer.hdr` gefolgt von `make` aus. Der Effekt sollte sein, dass `lexer.o`, `xtest_lexer.o` und `xtest_parser.o` im Unterverzeichnis `obj` neu erzeugt werden (die zugehörigen Source-Files binden `lexer.hdr` ein) und dann alle Testprogramme neu gelinkt werden.
6. Fügt das Makefile zum Repository hinzu (mit `git add`), committed und pusht es.

Schritt 3

Aktuell enthält `xtest_parser.abc` Code für den Parser, den Code-Generator und ist gleichzeitig ein Testprogramm. In diesem Schritt soll das besser aufgeteilt werden in die Dateien `parser.hdr`, `parser.abc`, `gen.hdr`, `gen.abc` und `xtest_parser.abc`:

- `gen.hdr` soll nur die (extern) Deklaration der Funktion `getReg` enthalten, und `gen.abc` die Implementierung.
- `parser.hdr` soll nur die (extern) Deklaration der Funktion `parseExpr` enthalten, und `parser.abc` soll nur das enthalten, was für die Implementierung von `parseExpr` notwendig ist.
- `xtest_parser.abc` soll nur den eigentlichen Test enthalten, also die bisherige Funktion `main` und alle notwendigen Include-Direktiven.

Bemerkung: Meine Vorgehensweise bei diesem Prozess ist, dass ich `xtest_parser.abc` kopiere mit:

```
cp xtest_parser.abc gen.hdr
cp xtest_parser.abc gen.abc
cp xtest_parser.abc parser.hdr
cp xtest_parser.abc parser.abc
```

und dann aus den Kopien und `xtest_parser.abc` lösche, was nicht benötigt wird.

Schritt 4

Mit `git status` könnt ihr anzeigen lassen, welche Dateien im Repository verändert wurden. Es wird auch angezeigt, welche Dateien noch nicht zum Repository gehören. Bei den nicht zugehörigen Dateien werden auch alle ausführbaren Dateien aufgelistet. Um zu überblicken, welche Source-Files noch nicht im Repository sind und hinzugefügt werden sollten (mit `git add FILE`), kann es sich lohnen, erst `make clean` und dann `git status` auszuführen.

Fügt alle neuen Source- und Header-Files zum Repository hinzu, committed und pushed sie.

Schritt 5

Aktuell werden im Struct `Expr` die Dezimal-Literale als Integer gespeichert. Für die Verwendung im Compiler wird es aber vorteilhaft sein, diese als Strings zu speichern. Wir werden später eine elegante Klasse für Strings schreiben. Vorerst werden wir für Strings einfach ein Array mit fester Länge verwenden und dafür in einer Headerdatei `string.hdr` einen Typ deklarieren mit:

```
type String: array[42] of char;
```

Aufgaben

1. Erstellt den Header `string.hdr`
2. Passt in `lexer.hdr` das Struct `Token` so an, dass das Member `val` diesen Typen verwendet. Bestätigt, dass in `lexer.abc` der Code nicht verändert werden muss, da sich nur die Dimension des Arrays verändert hat und die Dimension im Code in weiser Voraussicht immer mit

```
sizeof(token.val) / sizeof(token.val[0])
```

bestimmt wird.

3. Passt in `expr.hdr` das Struct `Expr` so an, dass das Member `decimal` diesen Typen verwendet. Geht die Implementierung von `expr.abc` durch und passt den Code an den Stellen an, wo dies notwendig ist.
4. Passt das Testprogramm `xtest_expr.abc` an. Zum Beispiel sollte jetzt ein Integer-Knoten mit `createIntegerExpr((String)"2")` erzeugt werden.
5. Wenn alles wieder funktioniert, aktualisiert das Repository, d.h. neue Dateien hinzufügen, committen und pushen.

Schritt 6

Der Parser soll jetzt nicht mehr sofort den Assembler-Code erzeugen, sondern einen Expression-Tree aufbauen. Der Assembler-Code kann dann später aus diesem Expression-Tree erzeugt werden, indem man eine Funktion analog zu `printExprTree` implementiert, aber das ist vorerst Zukunftsmusik. Ändert zunächst `parser.hdr` ab in:

```
@ "expr.hdr"
```

```
extern fn parseExpr(): -> Expr;
```

Diese Änderung wird jetzt eine Reihe von Anpassungen notwendig machen. Die einfachste davon ist im Testprogramm. Kann ein Ausdruck geparkt werden, dann liefert der Parser einen gültigen Zeiger auf einen Expression-Tree zurück, der ausgegeben werden kann. Ansonsten soll ein Null-Zeiger zurückgegeben werden.

Das Testprogramm kann also abgeändert werden in:

```
@ <stdio.h>
@ "gen.hdr"
@ "lexer.hdr"
@ "parser.hdr"

fn main()
{
    getToken();
    local expr: -> Expr = parseExpr();
    if (expr && token.kind == EOI) {
        printExprTree(expr);
        printf("\n");
    } else {
        printf("syntax error\n");
    }
}
```

Mit `make obj/xtest_parser.o` könnt ihr bereits jetzt prüfen, ob sich das Testprogramm übersetzen lässt. Damit kann man hier schon Tippfehler und fehlende Includes ausschließen.

In `parser.abc` muss natürlich mehr angepasst werden. Ihr sollt selber herausfinden, was zu machen ist. Dennoch ein paar Tipps:

- Alle Parse-Funktionen haben jetzt den Rückgabe-Typ `-> Expr`.
- Statt mit `printf`-Anweisungen Code zu erzeugen, wird jetzt ein passender Knoten angelegt. Und statt dem Wert eines Registers oder `-1` wird jetzt ein gültiger Zeiger oder `nullptr` zurückgegeben. Wird zum Beispiel in `parseFactor` ein Dezimal-Literal gefunden, kann dazu passender Code so aussehen:

```
local expr: -> Expr = createIntegerExpr(token.val);
getToken();
return expr;
```

- In den Parse-Funktionen für Binary-Expressions vereinfacht sich der Code teilweise sogar. Ist zum Beispiel `left` und `right` jeweils ein gültiger Zeiger und wurde in `parseTerm` ein `ASTERISK`-Token gefunden, kann man mit:

```
left = createBinaryExpr(EXPR_MUL, left, right);
```

einen neuen Knoten für die Multiplikation erzeugen und `left` auf diesen zeigen lassen. Damit man `left` für den Rückgabewert oder die nächste Iteration verwenden kann.

Wenn alles wieder klappt (also auch die Expression-Trees mit Latex visualisiert werden können) aktualisiert wieder das Repository.

Schritt 7

Um auch Subtraktion und Division zu unterstützen, kann die Grammatik erweitert werden zu:

```
expr = term { ( "+" | "-" ) term }
term = factor { ( "*" | "/" ) factor }
factor = decimal-literal | "(" expr ")"
```

Mit dieser Änderung hat die Subtraktion den gleichen Vorrang wie die Addition und die Division den gleichen Vorrang wie die Multiplikation.

Bemerkung zur EBNF-Notation:

- Mit geschweiften Klammern `{ }` wird eine Gruppe notiert, die beliebig oft (auch keinmal) wiederholt vorkommen kann.
- Mit runden Klammern `( )` wird eine nicht-optionale Gruppe notiert.
- Mit eckigen Klammern `[ ]` wird eine optionale Gruppe notiert, die einmal oder keinmal vorkommen kann.

Um die Grammatik umzusetzen, ist es notwendig, den Lexer, den Parser und die Datenstruktur für Expression-Trees anzupassen. Hier ist es hilfreich, sich eine Vorgehensweise zu überlegen, mit der man nach jeder Anpassung testen kann, ob diese funktioniert.

Wenn man bedenkt, dass der Parser den Lexer und den Expression-Tree benötigt, macht es Sinn, den Parser als letztes anzupassen. Man kann zum Beispiel so vorgehen:

1. **Testet**, was `xtest_lexer` bei der Eingabe von `-` und `/` in der aktuellen Fassung liefert. Das sollte jeweils ein `BAD_TOKEN` erzeugen. Fügt für `TokenKind` die Konstanten `MINUS` und `SLASH` hinzu und unterstützt sie im Code von `lexer.abc`. Dabei müsst ihr euch nochmals in `lexer.abc` einarbeiten und schauen, an welchen Stellen etwas angepasst oder ergänzt werden muss. Erst wenn mit `xtest_lexer` das neue Feature erfolgreich getestet wurde, den nächsten Teilschritt durchführen.

Pro-Tipp: Nach Stellen mit `PLUS` suchen, Code kopieren und für `MINUS` anpassen. Analog für `ASTERISK` und `SLASH` vorgehen.

2. **Expression-Trees anpassen:** Beginnt mit dem Testprogramm und ersetzt `EXPR_ADD` durch `EXPR_SUB` sowie `EXPR_MUL` durch `EXPR_DIV`, sodass die zwei neuen Knotentypen verwendet werden und nichts groß umgeschrieben werden muss. Fügt die Enum-Konstanten hinzu. Wenn man das schön alphabetisch macht, dann sieht das so aus:

```
enum ExprKind
{
    EXPR_ADD,
    EXPR_DIV,
    EXPR_MUL,
    EXPR_SUB,

    EXPR_INTEGER,
};
```

Überprüft im bisherigen Code, ob der Knoten ein Binary-Node ist, indem man die Gelegenheit nutzt, das Design für weitere Ergänzungen in der Zukunft robuster zu machen. Statt dem jetzt fehlerhaften Code

```
expr->kind >= EXPR_ADD && expr->kind <= EXPR_MUL
```

kann man mit nachfolgender Anpassung den robusten Code

```
expr->kind >= EXPR_BINARY && expr->kind < EXPR_BINARY_END
```

verwenden. Beachte, dass dabei ein `<` benutzt wird im Vergleich mit `EXPR_BINARY_END`.

Die Anpassung sieht so aus:

```
enum ExprKind
{
    EXPR_BINARY,
    EXPR_ADD = EXPR_BINARY,
    EXPR_DIV,
    EXPR_MUL,
    EXPR_SUB,
    EXPR_BINARY_END,

    EXPR_PRIMARY = EXPR_BINARY_END,
    EXPR_INTEGER = EXPR_PRIMARY,
    EXPR_PRIMARY_END,
};
```

Nach dieser Anpassung sollte es nicht weiter schwer sein, die restlichen Änderungen in `expr.abc` durchzuführen.

3. Parser anpassen:

Passt nun `parser.abc` an. Beachtet, dass nach `getToken` der Wert von `token` natürlich überschrieben wird. Braucht man den Wert später noch, dann kann man sich vorher eine Kopie anlegen:

```
while (token.kind == ASTERISK || token.kind == SLASH) {
    local tok: Token = token;
    getToken();
    // ...
}
```

Wenn alles wieder funktioniert (also auch die Expression-Trees mit Latex visualisiert werden können), aktualisiert wieder das Repository.

Schritt 8 (Parser: Code Refactoring)

Bei mir sieht die Implementierung von `parseTerm` nun so aus:

```
28 fn parseTerm(): -> Expr
29 {
30     local left: -> Expr = parseFactor();
31     if (!left) {
32         return nullptr;
33     }
34     while (token.kind == ASTERISK || token.kind == SLASH) {
35         local tok: Token = token;
36         getToken();
37         local right: -> Expr = parseFactor();
38         if (!right) {
39             printf("%u.%u: factor expected\n", token.pos.row, token.pos.col);
40             return nullptr;
41         }
42         local exprKind: ExprKind = tok.kind == ASTERISK ? EXPR_MUL : EXPR_DIV;
43         left = createBinaryExpr(exprKind, left, right);
44     }
45     return left;
46 }
```

Der Code für `parseExpr` sieht ähnlich aus. Was mich in beiden Fällen stört, ist, wie die Art eines Tokens (`ASTERISK` oder `SLASH`) auf die zugehörige Knotenart einer Expression abgebildet wird. Hier wird zum Beispiel in Zeile 35 erst in einer lokalen Variable das Token mit dem Operator in einer Variable `tok` abgespeichert, dann wird in Zeile 42 abhängig von `tok` eine Variable `exprKind` mit der zugehörigen Knotenart für die Expression initialisiert. Diese zwei Dinge gehören zusam-

men und sind für meinen Geschmack deshalb zu weit voneinander entfernt, und das führt aus meiner Erfahrung früher oder später zu Problemen.

Um den Code robuster zu gestalten, kann man eine (interne) Hilfsfunktion `exprKindFromToken` schreiben, die zu einem Token die passende Knotenart liefert (oder mit einer Assertion das Programm abbricht). Ergänzt dazu folgende Funktion und verwendet sie in `parseTerm` und `parseExpr`:

```
fn exprKindFromToken(token: Token): ExprKind
{
    switch (token.kind) {
        case ASTERISK: return ExprKind.MUL;
        // ...
        default:
            assert(0 && "internal error: exprKindFromToken");
            return ExprKind.UNKNOWN;
    }
}
```

Bemerkung: Ihr werdet feststellen, dass die Funktionen `parseTerm` und `parseExpr` nun bis auf wenige Stellen gleich aussehen. Wir werden das später ausnutzen, um den Code noch weiter zu vereinfachen, indem wir daraus eine einzige Funktion machen. Der Vorteil wird dann sein, dass man mit wenig Änderungen weitere (Binär-)Operatoren in der Grammatik unterstützen kann.

Vorbemerkung für die weiteren Schritte

Das Ziel ist, dass aus den Expression-Trees, die der Parser erzeugt, Assembler-Code generiert werden kann. In der ersten Version des Parsers hat dieser direkt den Assembler-Code erzeugt. Wir haben den Code in gewisser Hinsicht zwar verbessert, aber die Funktionalität verringert. Wir versuchen jetzt also, das wieder zu unterstützen, was früher schon einmal geklappt hat, aber mit einem besseren Design, das leichter zu erweitern und zu warten ist.

Aktuell gibt es die Funktion `printExprTree`, mit der man aus einem Expression-Tree Latex-Code erzeugen kann. Analog wird es eine Funktion `loadExpr` geben, die Assembler-Code erzeugt. Um diese Funktion "elegant" implementieren zu können, wird jetzt aber etwas Vorarbeit notwendig sein. In `gen.hdr` soll ein abstraktes Interface für die Code-Generierung zur Verfügung gestellt werden, das man später für `loadExpr` verwenden kann.

Schritt 9 (Interface für den Code-Generator und ein Testprogramm)

Man muss sich zunächst klar machen, dass der Code-Generator im Wesentlichen nur Funktionen implementiert, um Text auszugeben. Nur dass dieser Text eben Assembler-Code darstellt. Es wird zum Beispiel Funktionen geben, die Assembler-Code ausgeben, mit dem eine Konstante in ein Register geladen werden kann, oder mit dem Register addiert werden können. Zusätzlich erlaubt es der Code-Generator aber auch zu verwalten, welche Register aktuell zur Verfügung stehen.

Aber schauen wir uns zunächst nur die neue Version von `gen.hdr` an:

```
1 type GenReg: unsigned;
2
3 extern fn genReleaseReg(reg: GenReg);
4
5 extern fn genConstant(val: -> char): GenReg;
6
7 enum GenInstr
8 {
9     GEN_ADD,
10    GEN_SUB,
11    GEN_MUL,
12    GEN_DIV,
13 };
14
15 extern fn genInstr(instr: GenInstr, left: GenReg, right: GenReg): GenReg;
```

In Zeile 1 wird der Typ `GenReg` als Integer-Typ vereinbart. Das kann sich später noch ändern, so dass `GenReg` zum Beispiel ein Pointer-Typ oder Struct-Typ sein wird. Im Zusammenspiel mit dem

Code-Generator sollte man also immer `GenReg` verwenden und sich nicht auf die aktuelle Vereinbarung in Zeile 1 verlassen.

Die Bedeutung der Funktion `genReleaseReg` möchte ich erst nach der Bedeutung von `genConstant` und `genInstr` genauer erklären:

- Mit `genConstant` kann man Assembler-Code erzeugen, der eine Konstante in ein Register lädt. Der Aufruf von `genConstant("123")` könnte zum Beispiel folgenden Text ausgeben:

```
load 123, %1
```

also Assembler-Code, um die Konstante 123 in Register `%1` zu laden. Die Funktion gibt außerdem das Ziel-Register zurück, also in diesem Fall den Wert `1`.

- Mit `genInstr` kann man Assembler-Code erzeugen, um Register zu addieren, zu subtrahieren, zu multiplizieren oder zu dividieren. Die Art der Operation kann durch eine entsprechende Enum-Konstante angegeben werden. Der Aufruf von `genInstr(GEN_ADD, 1, 2)` könnte folgenden Text ausgeben:

```
add %1, %2, %3
```

und auch diese Funktion gibt das Ziel-Register zurück, also in diesem Fall den Wert `3`.

Intern wird im Code-Generator verwaltet, welche Register bei der Erzeugung von Assembler-Code bereits verwendet wurden, und ein bereits verwendetes Register wird nicht nochmals verwendet, es sei denn, es wird mit der Funktion `genReleaseReg` freigegeben.

Ein einfaches Testprogramm, das sich nicht darauf verlässt, dass Register mit einem Integer repräsentiert werden, könnte so aussehen:

```
1 @ "gen.hdr"
2
3 fn main()
4 {
5     local r1: GenReg = genConstant("12");
6     local r2: GenReg = genConstant("34");
7     local r3: GenReg = genInstr(GEN_ADD, r1, r2);
8     genReleaseReg(r3);
9     r3 = genInstr(GEN_SUB, r1, r2);
10    genReleaseReg(r1);
11    genReleaseReg(r2);
12    r1 = genConstant("42");
13    genReleaseReg(r1);
14 }
```

In Zeile 5 soll Code erzeugt werden, um den Wert `12` in ein Register zu laden, und in Zeile 6 Code, um den Wert `34` in ein anderes Register zu laden. In Zeile 7 soll Code erzeugt werden, um diese Register zu addieren und in ein weiteres, noch unbenutztes Register zu laden. In Zeile 8 wird das letzte Register (mit dem Ergebnis der Addition) wieder freigegeben und steht damit in Zeile 9 wieder zur Verfügung, wenn eine weitere Instruktion erzeugt wird. Und so weiter.

Aufgabe: Schreibt `gen.hdr` und `xtest_gen.abc`. Die Implementierung von `gen.abc` erfolgt erst im nächsten Schritt, ihr könnt also noch keine ausführbare Datei erzeugen. Aber ihr könnt mit

```
abc -c xtest_gen.abc
```

oder

```
make obj/xtest_gen.o
```

bereits ein Objekt-File erzeugen und damit überprüfen, dass es keine Tippfehler oder vergessene Includes gibt.

Schritt 10 (Implementierung des Code-Generator)

In `gen.abc` gibt es zusätzlich zu den im Header deklarierten Funktionen noch ein globales Array `usedReg` und eine interne Funktion `getReg`. Mit dem Array wird festgehalten, welche Register bereits benutzt wurden, und mit `getReg` kann man ein unbenutztes Register anfordern. Das Array ist ein simples Array von Booleans. Ist ein Eintrag `true`, dann ist es in Benutzung, und ansonsten steht es zur Verfügung. Mit `getReg` bekommt man das erste Register größer Null (da Register 0 eine Sonderbedeutung hat), das zur Verfügung steht. Mit `genReleaseReg` wird ein Array-Eintrag einfach auf `false` gesetzt. Eine einfache Implementierung kann also so aussehen:

```

1 @ <stdio.h>
2
3 @ "gen.hdr"
4
5 global usedReg: array[16] of bool;
6
7 fn getReg(): GenReg
8 {
9     for (local r: GenReg = 1; r < sizeof(usedReg)/sizeof(usedReg[0]); ++r) {
10         if (!usedReg[r]) {
11             usedReg[r] = true;
12             return r;
13         }
14     }
15     assert(0 && "getReg: out of registers!\n");
16     return 0;
17 }
18
19 fn genReleaseReg(reg: GenReg)
20 {
21     assert(usedReg[reg]);
22     usedReg[reg] = false;
23 }

```

Zur Verfügung stehen insgesamt die Register `%1` bis `%15`. Mit geeigneten Assertions wird hier noch überprüft, dass `getReg` ein freies Register gefunden hat und in `genReleaseReg` auch tatsächlich ein benutztes Register freigegeben wird.

Das war der komplizierteste Teil. Die weiteren Funktionen generieren im Wesentlichen nur Text und geben ein zuvor mit `getReg` allokiertes Zielregister zurück. Zum Beispiel kann `getConstant` so implementiert werden:

```

25 fn genConstant(val: -> char): GenReg
26 {
27     local reg: GenReg = getReg();
28     printf("\tload \t%s, \t%%d\n", val, reg);
29     return reg;
30 }

```

Aufgabe

Implementiert die Funktion `genInstr`. Dabei sollen alle möglichen Werte von `instr` abgedeckt werden und folgendes Realität werden:

- Der Aufruf von `genInstr(GEN_ADD, r1, r2)` erzeugt Assembler-Code der Form:
`add %1, %2, %3`
- Der Aufruf von `genInstr(GEN_SUB, r1, r2)` erzeugt Assembler-Code der Form:
`sub %1, %2, %3`
- Der Aufruf von `genInstr(GEN_MUL, r1, r2)` erzeugt Assembler-Code der Form:
`mul %1, %2, %3`
- Der Aufruf von `genInstr(GEN_DIV, r1, r2)` erzeugt Assembler-Code der Form:
`div %1, %2, %3`

Testet die Implementierung mit `xtest_gen`. Hier kann und sollte man den Test eigentlich anpassen, so dass alle Operationen ausgeführt werden. Ich selbst wäre dafür aber jetzt wahrscheinlich zu faul und würde mit dem nächsten Schritt weitermachen und Copy-Paste-Fehler erst dann korrigieren, wenn mir diese im nächsten Schritt auffallen.

Am Ende das Repository aktualisieren, stellt mit git Status sicher, dass auch alles drin ist.

Für die folgende freie Fläche ist ein Baum gestorben. Rechtfertigen dies durch wichtige Notizen: